



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
 ESCUELA DE INGENIERÍA
 DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2223 — Teoría de Autómatas y Lenguajes Formales — 2' 2025
 IIC2224 — Autómatas y Compiladores — 2' 2025

PAUTA INTERROGACIÓN 2

Pregunta 1

Dado $w = w_1 \dots w_m \in \Sigma^*$, recuerde que $A_w = (\{0, \dots, m\}, \Sigma, \Delta, \{0\}, \{m\})$ es el autómata tal que $\Delta = \{(0, a, 0) \mid a \in \Sigma\} \cup \{(i, w_{i+1}, i+1) \mid i < m\}$. Sea $A_w^{\det} = (Q^{\det}, \Sigma, \delta^{\det}, \{0\}, F^{\det})$ la determinización de A_w tal que $Q^{\det}$ contiene solo los estados alcanzables desde $\{0\}$.

Demuestre que para todo $S \in Q^{\det}$ y $i \in \{0, 1, \dots, m\}$ se cumple que $i \in S$ si, y solo si, $w_1 \dots w_i$ es un sufijo de $w_1 \dots w_{\max(S)}$.

Solución

Sea $S \in Q^{\det}$ alcanzable. Entonces existe una palabra $u = a_1 a_2 \dots a_k \in \Sigma^*$ tal que

$$\widehat{\delta}^{\det}(\{0\}, u) = S.$$

Por definición de la determinización, se tiene que $j \in S$ si y solo si existe una ejecución en A_w sobre u que termina en el estado j . Recordando la estructura de A_w , cualquier ejecución que termine en j tiene la forma

$$0 \xrightarrow{a_1} 0 \xrightarrow{a_2} 0 \dots \xrightarrow{a_{k-j}} 0 \xrightarrow{w_1} 1 \xrightarrow{w_2} 2 \dots \xrightarrow{w_j} j.$$

De esto se sigue inmediatamente que $w_1 \dots w_j$ es sufijo de u . En particular, como $\max(S) \in S$, también $q = w_1 \dots w_{\max(S)}$ es sufijo de u . Por lo tanto, existe $x \in \Sigma^*$ tal que

$$u = xq.$$

Definamos además $p = w_1 \dots w_i$.

$\Rightarrow$. Supongamos $i \in S$. Entonces, como antes, p también es sufijo de u , es decir, existe $y \in \Sigma^*$ tal que

$$u = yp.$$

Como q también es sufijo de u , al comparar las porciones finales de u se obtiene que q termina en p . Es decir, existe $z \in \Sigma^*$ tal que

$$q = zp.$$

Por lo tanto, p es un sufijo de q , como queríamos.

$\Leftarrow$. Supongamos ahora que p es sufijo de q , es decir, que $q = zp$ para algún $z \in \Sigma^*$. Como hemos visto, q es sufijo de u . Por la transitividad de la relación “ser sufijo”, se concluye que p también es sufijo de u . De

nuevo, por corrección de la determinización, si p es sufijo de u entonces existe una ejecución en A_w sobre u que termina en el estado i , es decir, $i \in S$.

Habiendo probado ambas implicancias, entonces queda probada la equivalencia.

Distribución de puntaje:

- (2 ptos.) Por el sufijo de w .
- (2 ptos.) La implicancia hacia la izquierda.
- (2 ptos.) La implicancia hacia la derecha.

Pregunta 2

Para poder construir una gramática libre de contexto en forma normal de Chomsky para $\mathcal{L}(\mathcal{G}) \setminus \{\epsilon\}$ debemos seguir los siguientes pasos:

1. Eliminación de producciones unitarias $X \rightarrow Y$. Antes de eliminarlas, si existen las reglas $X \rightarrow Y$ y $Y \rightarrow \gamma$, entonces debemos incluir $X \rightarrow \gamma$. A partir de lo anterior obtenemos:

$$\mathcal{G} : \begin{array}{l} S \rightarrow aSb \mid T \mid bTa \mid S \mid \epsilon \\ T \rightarrow bTa \mid S \mid \epsilon \mid aSb \mid T \end{array}$$

Luego eliminamos todas las producciones unitarias obteniendo,

$$\mathcal{G} : \begin{array}{l} S \rightarrow aSb \mid bTa \mid \epsilon \\ T \rightarrow bTa \mid \epsilon \mid aSb \end{array}$$

2. Eliminación de producciones vacías $X \rightarrow \epsilon$. Antes de eliminarlos, si existen las reglas $Z \rightarrow \alpha X \beta$ y $X \rightarrow \epsilon$, entonces debemos incluir $Z \rightarrow \alpha \beta$. Luego,

$$\mathcal{G} : \begin{array}{l} S \rightarrow aSb \mid bTa \mid \epsilon \mid ab \mid ba \\ T \rightarrow bTa \mid \epsilon \mid aSb \mid ba \mid ab \end{array}$$

Después eliminamos las producciones vacías y como resultado obtenemos la siguiente gramática equivalente sin producciones inútiles,

$$\mathcal{G} : \begin{array}{l} S \rightarrow aSb \mid bTa \mid ab \mid ba \\ T \rightarrow bTa \mid aSb \mid ba \mid ab \end{array}$$

- * Reducción de la gramática. Comparando las producciones de S y T podemos simplificar el problema dejando la gramática de la siguiente forma (Este paso no era necesario, pero simplificaba bastante el problema).

$$\mathcal{G} : \begin{array}{l} S \rightarrow aSb \mid bSa \mid ab \mid ba \end{array}$$

3. Agregar nuevas variables $A \rightarrow a$, para cada símbolo $a \in \Sigma$ y remplazar las ocurrencias de a por A .

$$\mathcal{G} : \begin{array}{l} S \rightarrow ASB \mid BSA \mid AB \mid BA \\ A \rightarrow a \\ B \rightarrow b \end{array}$$

4. Agregar $Z \rightarrow Y_2 \dots Y_k$ y $X \rightarrow Y_1 Z$ si existe en la gramática una producción $X \rightarrow Y_1 Y_2 \dots Y_k$ $k \geq 3$ con el objetivo de eliminar producciones de tres o más variables.

$$\mathcal{G} : \begin{array}{l} S \rightarrow AX_a \mid BX_b \mid AB \mid BA \\ X_a \rightarrow SB \\ X_b \rightarrow SA \\ A \rightarrow a \\ B \rightarrow b \end{array}$$

Finalmente, la gramática anterior representa la forma normal de Chomsky para $\mathcal{L}(\mathcal{G}) \setminus \{\epsilon\}$.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(1.5 puntos)** Por la eliminación de producciones unitarias.
- **(1 punto)** Por la eliminación de producciones vacías.
- **(0.5 puntos)** Por obtener una gramática equivalente sin producciones inútiles (unitarias y vacías).
- **(1 punto)** Por agregar las variables A y B del paso 3.
- **(2 punto)** Por agregar las variables X_a y X_b del paso 4.

Pregunta 3

Usando el teorema de Myhill-Nerode demuestre que el siguiente lenguaje no es regular:

$$L = \{ a^n b^m \mid n \text{ es par o } n < m \}$$

Solución

Primero recordemos que $\equiv_L$ es una relación que cumple lo siguiente:

$$u \equiv_L v \text{ ssi } (u \cdot w \in L \iff v \cdot w \in L) \forall w \in \Sigma^*.$$

Para demostrar que L no es regular, tenemos que demostrar que la relación $\equiv_L$ tiene una cantidad **infinita** de clases de equivalencia.

Considere la familia infinita de palabras

$$S = \{ a^{2k+1} \mid k \in \mathbb{N} \},$$

todas con número impar de a 's. Probaremos que cualesquiera dos palabras distintas de S son distinguibles por $\equiv_L$.

Sean $i, j \in \mathbb{N}$ con $i \neq j$, y tomamos las dos palabras

$$x = a^{2i+1} \quad \text{y} \quad y = a^{2j+1}.$$

Sin pérdida de generalidad, vamos a asumir que $i < j$, si fuera al contrario la demostración sería analoga. Tomemos la siguiente palabra

$$w = b^{2i+2}.$$

Ahora tenemos lo siguiente:

- $xw = a^{2i+1}b^{2i+2} \in L$, porque el número de a 's es $2i+1$ (impar) y el de b 's es $2i+2$, de modo que se cumple la condición $n < m$
- $yw = a^{2j+1}b^{2i+2} \notin L$. Como $j > i$ se tiene

$$j \geq i+1 \Rightarrow 2j \geq 2i+2 \Rightarrow 2j+1 \geq 2i+3 > 2i+2,$$

por lo que no se cumple $n < m$. Tampoco se cumple " n es par", pues $2j+1$ es impar. Por tanto $yw \notin L$.

Entonces concluimos que $\equiv_L$ tiene infinitas clases de equivalencia que se ven de la siguiente forma:

$$\llbracket a^{2n+1} \rrbracket_{n \in \mathbb{N}}$$

Concluimos por el teorema de Myhill-Nerode que L **no** es regular.

Pauta de corrección

- **(0.5 punto)** Enunciar correctamente Myhill-Nerode y la estrategia (infinitas clases $\Rightarrow$ no regular).
- **(2 puntos)** Construir la familia $S = \{a^{2k+1}\}$ o alguna familia parecida que funcione.
- **(0.5 punto)** Por tomar dos palabras distinguibles entre sí de la familia.
- **(1 puntos)** Por elegir $w = b^{2i+2}$ o un w corrector para la familia elegida.
- **(1 punto)** Por concluir que una palabra está en L .
- **(1 punto)** Por concluir que la otra palabra no está en L .

Pregunta 4

Para la resolución de este problema utilizaremos una tabla de hash $H : Q \rightarrow \mathbb{N}$, donde $H[q]$ almacena la cantidad de posiciones desde donde se puede llegar hasta ese estado. Además, podemos suponer que la tabla de hash nos permite acceder a los valores guardados en tiempo $\mathcal{O}(1)$.

Considerando lo anterior, el siguiente pseudocódigo resuelve el problema pedido:

Algoritmo: Count-All

Input: Un DFA- $\mathcal{A} = \{Q, \Sigma, \delta, I, F\}$ y una palabra $w = a_1 \dots a_n \in \Sigma^*$.

Output: $|\{(i, j) \mid i \leq j \wedge a_i \dots a_j \in \mathcal{L}(\mathcal{A})\}|$.

Function Count-All($\mathcal{A}, w$):

```

 $H \leftarrow \emptyset, c \leftarrow 0$ 
for  $j = 1$  to  $n$  do
     $H' \leftarrow H$ 
     $H \leftarrow \emptyset$ 
    foreach  $p \in Q$  do
         $q \leftarrow \delta(p, a_j)$ 
         $H[q] \leftarrow H[q] + H'[p]$ 
    end
     $q \leftarrow \delta(q_0, a_j)$ 
     $H[q] \leftarrow H[q] + 1$ 
    foreach  $p \in F$  do
         $c \leftarrow c + H[p]$ 
    end
end
return  $c$ ;

```

En palabras, lo que hacemos es mantener un conjunto de estados en las llaves de H de tal manera que $H[q]$ es el número de posiciones i tal que la ejecución de $\mathcal{A}$ desde i llega hasta q . En cambio, la variable c recuerda el número de pares (i, j) tal que $a_i \dots a_j \in \mathcal{L}(\mathcal{A})$ encontradas hasta este momento. Esto lo podemos formalizar con la siguiente invariante:

- **Invariante:** $Inv(j)$; Al terminar la j -ésima iteración, $\forall q \in Q$,

$$H[q] = |\{a_i \dots a_j \mid \delta^*(q_0, a_i \dots a_j) = q\}|$$

es decir, $H[q]$ almacena el número total de subcadenas $a_i \dots a_j$ tal que llegan al estado q ; y

$$c = |\{a_i \dots a_k \mid k \leq j \wedge a_i \dots a_k \in \mathcal{L}(\mathcal{A})\}|$$

es decir, c es un contador que almacena la cuenta acumulada de todas las subcadenas que son aceptadas por $\mathcal{A}$ hasta la posición j .

Así, podemos notar que al final de la iteración n , c va a contener el número acumulado de todas las subcadenas $a_i \dots a_n$ tal que son aceptadas por $\mathcal{A}$, lo cual es lo que nos pide como *output* el algoritmo.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(1 punto)** Por la idea de usar la tabla de Hash $H : Q \rightarrow \mathbb{N}$ y un contador, o una estructura parecida.
- **(1 punto)** Por inicializar H vacío, el contador en cero, y asegurarse de mantener $H[q] = 0$ en el inicio de cada iteración.

- **(1 punto)** Por considerar las ejecuciones tal que $q = \delta(p, a_j)$ y actualizar $H[q]$ de manera consecuente (primer **for each**).
- **(1 punto)** Por considerar las ejecuciones tal que $q = \delta(q_0, a_j)$ y actualizar $H[q]$ de manera consecuente, es decir, las cadenas de largo 1 y el $H[q] = H[q] + 1$.
- **(1 punto)** Por actualizar satisfactoriamente el contador (segundo **for each**).
- **(1 punto)** Por explicar la correctitud del algoritmo. Punto condicional a que el algoritmo esté correcto.